

# About the Product Development Framework & Hierarchy

## Purpose

This document defines the product development hierarchy and the four documents that hold the upstream context every team needs to build coherently. It exists so that a PM drafting a PRD, a GM scoping a new line of business, or an engineer trying to understand why a module exists can locate themselves without rebuilding the system from scratch.

## Why These Documents Exist

Implementation has gotten cheap. Code that used to take weeks now lands in minutes. The bottleneck has moved upstream from execution to planning: deciding what to build, why, and how it fits with everything else in the portfolio.

Upstream work doesn't compress the same way. Customer interviews take as long as customers take. Market research takes as long as the market takes to reveal itself. GMs commit to lines of business at the speed of board meetings and budget cycles. None of that gets faster because an agent can write a React component in five seconds. The pacing of strategy and requirements is set by the world, not by tooling.

That asymmetry is why these documents matter more now, not less. They are the durable substrate that feeds everything downstream. PRDs ladder up to Product Strategies. Product Strategies ladder up to Solution Strategies. At execution, the whole stack hands off to Jira, or whatever supersedes Jira as the live surface where humans, agents, and context meet.

These documents are written for two audiences at once. The humans on the team. And the agents that will increasingly help build, plan, and reason about the work. That changes how they should be written: short, structured, dense per section, with explicit links to the documents above and below. A document too long for a person to read is also too long for an agent to use well.

The documents are tiered by rate of change. Solution Strategy moves slowly because lines of business move slowly. Product/Module Strategy moves when strategic direction shifts. PRDs move through discovery and lock at dev start. Technical design lives alongside the build. Once execution begins, Jira owns current state. The strategy and requirements documents stay as the reference for what was agreed and why.

## The Hierarchy

Product work is organized into five tiers.

Visual: [LINK](#)

 Product Development Hierarchy



Could not identify user

| Tier              | Definition                                                                                                                                                                                                                                                 | Example                                                                                                                           |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Solution</b>   | A product or combination of products that delivers a distinct business outcome for a customer. What gets sold as a packaged offering. A solution is also a line of business: offering one is a strategic commitment to participate in a particular market. | Patient Engagement Solution (combines Scheduling and Communications products to deliver an end-to-end patient engagement outcome) |
| <b>Product</b>    | A branded, billable collection of capabilities serving specific personas.                                                                                                                                                                                  | EMR (for the provider persona)                                                                                                    |
| <b>Module</b>     | A billable unit within a product that delivers discrete value or solves a distinct problem.                                                                                                                                                                | Lab Results Skill (a module within a larger product)                                                                              |
| <b>Capability</b> | A set of features that together solve a problem or serve a specific need.                                                                                                                                                                                  | Automated Lab Results Delivery                                                                                                    |
| <b>Feature</b>    | A discrete mechanism that effects value-delivery action. The level where engineering work gets done.                                                                                                                                                       | Lab Results Automation Rules Configuration Page                                                                                   |

Solutions emerge from one or more products. Products contain modules. Modules deliver capabilities. Capabilities are enabled by features. Features are actioned by users or agents.

Every tier has an artifact that carries its context. Solutions, Products, and Modules are defined in the strategy and requirements documents this framework templatizes. Capabilities and features are defined in Jira: capabilities as epics, features as tickets. The cascade runs the full length of the hierarchy. The medium changes at the PRD-to-Jira handoff because the work changes from planning to execution, but the chain of context does not break.

Templates for the Jira artifacts (epic and ticket structure, conventions for shared context that humans and agents both pull from) are out of scope for this framework today. They are a likely future addition.

Orthogonal to the hierarchy, **shared services** (internal APIs, external integrations, data infrastructure, messaging services) support multiple tiers horizontally. They are the chassis the hierarchy runs on, not a tier in the hierarchy itself.

## The Four Documents

Each document answers a different question at a different tier. Detailed guidance for each one lives in that document's template. What follows is the framework view: just enough to know which document is which.

### Solution Strategy Document

The Why and What at the line-of-business level. It defines the customer outcome the solution delivers, the products and modules that compose it, the workflow that ties them together, and the dependencies the solution creates across the portfolio.

Owned by the GM responsible for the line of business. Senior product, sales, marketing, and finance leadership contribute. Read by executive leadership, the GM's organization, and every product team whose work is part of the solution.

Changes slowly. Minor updates when modules ship or composition shifts. Major updates only when the line of business itself is repositioned.

The Solution Strategy is the seam between the business case for a line of business (which lives in a pitch deck and a business plan) and the product strategies for the components that compose it. It does not duplicate either upstream artifact. It does the one thing neither does well: it gives every component team a shared understanding of what the solution is and what they must build, modify, or honor to make it real.

## **Product/Module Strategy Document**

The Why (scoped) and conceptual What. It defines why this product or module exists within the solutions it participates in, the concept of the thing in plain language, the personas it serves, and its strategic direction.

Owned by the product lead for that tier. Read by the team that owns it, adjacent teams, and the GM of any solution it participates in.

Changes when strategic direction shifts or the conceptual model changes. A product or module can participate in multiple solutions. The Product Strategy doesn't change based on which solution is being sold. It references each one for context.

Whether to call this "Product Strategy" or "Module Strategy" depends on the tier it covers. The document type is the same. The scope differs. Small modules may not need their own strategy and can reference the parent product's strategy directly.

## **PRD (Product Requirements Document)**

The What (specific). It defines the capabilities, behaviors, constraints, edge cases, and acceptance criteria for a single module. Detailed enough that engineers, designers, and agentic coding tools can build from it without guessing.

Owned by the PM for the module. Read by engineering, design, QA, and any team whose work depends on the module.

Revised through discovery and iteration. Locked when development begins, with subsequent changes tracked as explicit amendments. Once execution starts, Jira owns current state. The PRD remains the reference for what was agreed and why.

A well-written PRD is short because its upstream documents carry the context. A PRD that balloons into executive summaries and strategic framing is usually a symptom of missing documents above it in the hierarchy.

## **Technical Design Document**

The How. It defines the architecture, data models, API contracts, integration points, technical dependencies, and engineering approach for building what the PRD specifies.

Owned by the tech lead for the module, in partnership with the PM. Read by the engineering team building it, architecture review, and adjacent tech leads. Lives alongside the PRD through development.

## How the Documents Connect

Each document points upward for context and downward for specifics. The PRD references the Product/Module Strategy for why the module exists. The Product/Module Strategy references the Solution Strategy for the broader business context. The Technical Design references the PRD for what's being built. The business plan, a separate financial artifact owned by the GM or commercial team, references the Solution Strategy for what's being sold.

When a document is missing at a higher tier, the gap gets absorbed into the documents below it. Those documents then become bloated and unfocused, trying to carry context they were never designed to hold. Creating documents at the right tier keeps every artifact at its right size.

Two through-lines run vertically through the system. The **GM** is the through-line at the solution tier, carrying line-of-business accountability across every product and module that composes the solution. The **PM** is the through-line at the module tier, carrying the module from strategy through requirements and into partnership with engineering on technical design.

## Summary Table

| Artifact                | Tier              | Question                                              | Owner        | Changes                                |
|-------------------------|-------------------|-------------------------------------------------------|--------------|----------------------------------------|
| Solution Strategy       | Solution          | Why (line of business) + what's sold + product impact | GM           | Slowly                                 |
| Product/Module Strategy | Product or Module | Why (scoped) + conceptual what                        | Product lead | As direction shifts                    |
| PRD                     | Module            | What (specific requirements)                          | PM           | Through discovery, locked at dev start |
| Technical Design        | Module            | How (implementation)                                  | Tech lead    | Through build                          |

|             |            |                                        |                         |                   |
|-------------|------------|----------------------------------------|-------------------------|-------------------|
| Jira Epic   | Capability | What gets executed for this capability | PM and tech lead        | Live during build |
| Jira Ticket | Feature    | What gets executed for this feature    | Whoever owns the ticket | Live during build |